Assignment-04

1. Explain the concept of Column-Oriented NoSQL Databases. How does the column-family model differ from the traditional relational table model? Discuss the advantages and limitations of using column-oriented databases for large-scale applications.

Concept. A column-oriented (wide-column) NoSQL database stores and organizes data by columns grouped into *column families* rather than by fixed, uniform rows. Each row is identified by a unique key, and different rows in the same table may have completely different sets of columns. Data is physically partitioned by row key (or partition key) across many commodity servers, which makes the model ideal for massive, sparse, write-heavy datasets. Examples: Apache HBase, Apache Cassandra, Google Bigtable.

Column-family model vs. relational model

Aspect	Relational (RDBMS)	Column-family store
Schema	Fixed schema; every row has the same columns	Only column families are fixed; columns (qualifiers) are created dynamically per row
NULL values	Empty cells stored as NULL, wasting space	Absent columns take no space (sparse storage)
Storage	Row-wise on disk pages	Grouped by column family; each family stored in separate files
Relationships	Foreign keys and JOINs	No joins; data is denormalised and modelled around queries
Scaling	Mainly vertical; sharding is complex	Built-in horizontal scaling over commodity nodes
Transactions	Full ACID, multi-row	Limited (row-level atomicity in HBase; tunable consistency in Cassandra)
Versioning	Not built-in	Cells may keep multiple timestamped versions

Advantages

- Horizontal scalability to petabytes and thousands of nodes.
- Very high write throughput (log-structured writes: commit log/WAL + memtable).
- Flexible schema; efficient storage of sparse data.
- Efficient reads and compression because similar data is stored together.
- Built-in replication, fault tolerance and (in Cassandra) multi-datacenter support.

Limitations

- No joins, no ad-hoc queries; queries must be known in advance and the model designed around them.
- Weaker consistency / limited transactions compared with ACID.
- Data duplication through denormalisation increases storage and update complexity.
- Poor row-key or partition-key choices cause hotspots.
- Operational complexity (compaction, tuning, repair) and a steeper learning curve.

2. Compare Apache HBase and Apache Cassandra with respect to data model, architecture, consistency, availability, scalability, fault tolerance, query processing, and suitable applications. Which one would you choose for a highly distributed application and why?

Feature	Apache HBase	Apache Cassandra
Data model	Table - row key - column family - qualifier - timestamped cell; rows sorted by row key	Keyspace - table - partition key + clustering columns; CQL tables with typed columns

Feature	Apache HBase	Apache Cassandra
Architecture	Master-slave: HMaster, RegionServers, ZooKeeper, built on HDFS	Masterless peer-to-peer ring; every node equal; gossip protocol, no HDFS
Consistency	Strong consistency (CP): single RegionServer serves a row	Tunable (AP by default): eventual to strong via consistency levels (ONE, QUORUM, ALL)
Availability	Reduced during RegionServer/Master failover until regions are reassigned	Very high; no single point of failure, any replica can serve requests
Scalability	Scales by region splitting across RegionServers; depends on HDFS/ZooKeeper	Linear scaling by adding nodes; consistent hashing redistributes data
Fault tolerance	HDFS replication (default 3) + WAL replay; ZooKeeper-based master election	Replication factor per keyspace, hinted handoff, read repair, anti-entropy repair
Query processing	Get, Put, Scan, Delete (Java API/shell); range scans on row key; Hive/Phoenix for SQL	CQL (SQL-like) with partition-key-based queries; secondary indexes, materialized views
Multi-datacenter	Possible but replication is more complex	Native, first-class multi-DC replication
Suitable for	Hadoop analytics, random real-time read/write over huge tables, strict consistency (e.g., messaging, time-series with scans)	Always-on, globally distributed, write-heavy apps (IoT, logging, personalisation, session data)

Choice for a highly distributed application: Apache Cassandra. Its masterless design removes single points of failure, it offers native multi-datacenter replication, linear write scalability, and tunable consistency so each operation can trade consistency for latency/availability. HBase would be preferred only when strong consistency and tight Hadoop/HDFS integration are the main requirements.

3. Explain the architecture of Apache HBase in detail. Describe the roles and interactions of HMaster, RegionServer, ZooKeeper, HDFS, Regions, and MemStore during read and write operations. Draw a suitable architecture diagram.

HBase is a distributed, column-oriented database modelled on Google Bigtable and built on top of HDFS. A table is horizontally split into **Regions** (contiguous row-key ranges), and regions are served by **RegionServers**.

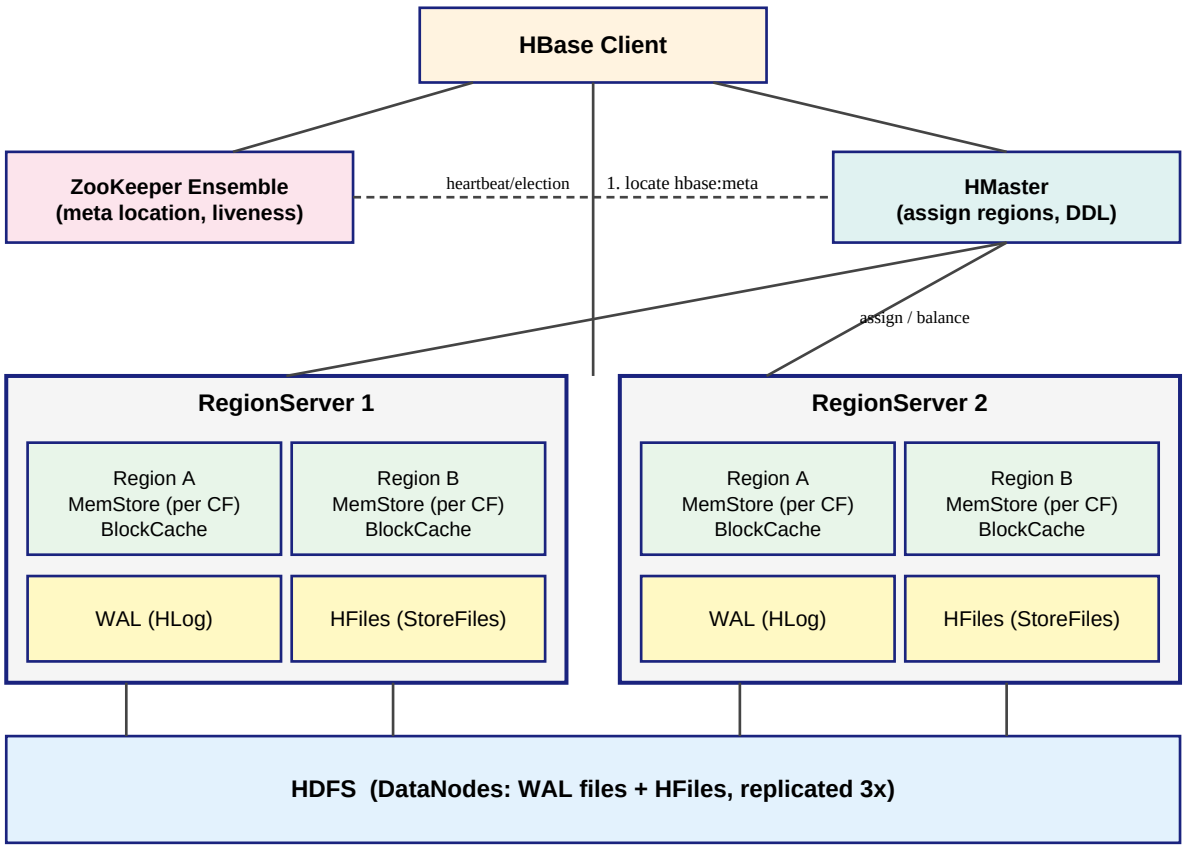

Figure: Apache HBase architecture.

Components

- **HMaster** - assigns regions to RegionServers, balances load, handles schema changes (create/alter/drop table), and reassigns regions of failed servers. It is not on the read/write data path.
- **RegionServer** - serves reads and writes for the regions it hosts; manages WAL, MemStore, BlockCache, flushes, compactions and region splits.
- **ZooKeeper** - coordination service: tracks live servers, elects the active HMaster, stores the location of the *hbase:meta* table.
- **HDFS** - durable, replicated storage for HFiles and WAL files.
- **Region** - a contiguous range of rows [start key, end key) of a table; splits automatically as it grows.
- **MemStore** - in-memory sorted write buffer, one per column family per region; flushed to an immutable HFile when full.

Write path

- Client asks ZooKeeper for the *hbase:meta* location, finds the RegionServer for the row key and caches it.
- The Put is first appended to the **WAL** on HDFS (durability).
- It is then written to the **MemStore** and acknowledged to the client.
- When the MemStore reaches its threshold it is flushed as a new sorted **HFile** to HDFS; minor/major compactions later merge HFiles.

Read path

- Client locates the RegionServer (ZooKeeper, meta table, cached).
- The RegionServer checks the **MemStore**, the **BlockCache**, and then the HFiles (using Bloom filters and indexes to skip files).
- Results from all sources are merged by timestamp and the latest version(s) returned to the client.

4. Explain the Column-Family Data Store model with the help of a suitable example. Discuss concepts such as row key, column family, column qualifier, cell, timestamp, and versioning. How is this model different from rows and columns in an RDBMS?

A column-family store is a sorted, sparse, multi-dimensional map: **(row key, column family:qualifier, timestamp) -> value**. Example: a *Student* table.

Row key	Family: personal	Family: academic	Family: activity
S1001	name = Anil; city = Meerut	course = BCA; cgpa = 8.2	login:2026-10-01 = web; event:e1 = Quiz
S1002	name = Riya	course = MCA; semester = 3; cgpa = 8.9 (ts=t2), 8.5 (ts=t1)	library:2026-10-02 = Book issued

- **Row key** - unique identifier; rows are stored sorted by it, so it controls data distribution and scan speed.
- **Column family** - logical/physical group of columns, defined at table creation (e.g., personal, academic). Stored together in separate files, with their own compression and TTL settings.
- **Column qualifier** - the column name inside a family (e.g., academic:cgpa). Created on the fly, so each row can have different qualifiers.
- **Cell** - the intersection of row key, family:qualifier and timestamp; holds an uninterpreted byte array value.
- **Timestamp** - a 64-bit value attached to every cell (default: write time) that identifies the version.
- **Versioning** - several versions of a cell can be retained (configurable max versions); a read returns the latest by default, and older versions can be requested by timestamp (e.g., cgpa 8.9 vs 8.5 above).

Difference from RDBMS: an RDBMS has a rigid schema where each row holds every column (NULLs where empty), one value per cell, and relations via joins. A column-family store has only families fixed, qualifiers dynamic per row, no NULL storage, multiple timestamped versions per cell, and rows ordered by key for distributed range access.

5. A university needs to store billions of student activity records, where different students may have different attributes. Design a suitable HBase data model for this application and justify your choice of row key and column families.

Table: `student_activity`

Row key design: `<salt(2 digits)>#<studentId>#<reverseTimestamp>` where reverseTimestamp = Long.MAX_VALUE - eventTime. Example: `07#S1001#9223370...`

Column family	Qualifiers (examples)	Purpose / settings
profile (p)	name, dept, year, email, city	Rarely changed student attributes; VERSIONS=3
academic (a)	course, semester, cgpa, result_<sem>	Academic events; compression SNAPPY
activity (act)	type, details, location, device, score, club_<name>	High-volume activity records; variable attributes per student; VERSIONS=1, optional TTL
library/attendance (l)	book_id, issue_date, attendance_<date>	Separate access pattern; separate files for efficient scans

Justification

- **Salt prefix** spreads writes over all regions and avoids hotspotting from sequential IDs.
- **studentId** keeps all records of one student adjacent, so a single prefix scan returns that student's history.
- **Reverse timestamp** makes the newest activity come first in scans.
- **Few column families** (HBase recommends 2-3) because each family has its own MemStore and HFiles; related columns that are read together are grouped, rarely-read data is separated.

- **Dynamic qualifiers** let each student have different attributes without schema change or NULL overhead.
- Sample operations: `put 'student_activity','07#S1001#...','act:type','Quiz'`; `scan ... STARTROW=>'07#S1001#'`.

6. Discuss consistency and transactions in HBase and Cassandra. Why do distributed NoSQL databases often make different trade-offs from traditional ACID-based relational databases? Explain how consistency can affect application design.

HBase

- Strongly consistent: each row belongs to exactly one region served by one RegionServer, so reads always see the latest committed write.
- Atomic operations at the **single-row** level (even across column families); checkAndPut and increment are atomic. No native multi-row ACID transactions (Apache Phoenix/Omid/Tephra add them).
- Durability through WAL on HDFS; MVCC gives readers a consistent snapshot.

Cassandra

- Tunable consistency per query: ONE, QUORUM, LOCAL_QUORUM, ALL, etc. R + W > RF gives strong consistency; lower levels give eventual consistency.
- Atomicity and isolation only at the partition (row) level; batches are atomic but not isolated across partitions.
- Lightweight transactions (LWT, Paxos-based `IF NOT EXISTS`) give linearizable compare-and-set at higher latency.
- Conflicts resolved by last-write-wins using timestamps; read repair and anti-entropy repair converge replicas.

Why different trade-offs from ACID?

The **CAP theorem** states that during a network partition a distributed system must choose between consistency and availability. Coordinating distributed ACID transactions (two-phase commit, global locks) costs latency, limits throughput and reduces availability. NoSQL systems therefore relax guarantees (BASE: Basically Available, Soft state, Eventually consistent) to gain scalability, low latency and fault tolerance.

Effect on application design

- Design operations around a single row/partition so atomicity is available.
- Make writes idempotent and tolerate duplicates/retries.
- Choose the consistency level per use case (QUORUM for payments, ONE for page views).
- Handle stale reads and conflicts in application logic; avoid read-modify-write patterns without LWT/checkAndPut.

7. Availability and fault tolerance are important characteristics of distributed databases. Explain how Apache Cassandra achieves high availability using its distributed architecture. Discuss the role of replication and consistency levels in maintaining availability.

- **Masterless ring:** all nodes are equal peers, so there is no single point of failure. Any node can act as coordinator for a request.
- **Consistent hashing / virtual nodes:** data is placed on the ring by hashing the partition key; vnodes spread load and make adding/removing nodes smooth.
- **Gossip protocol** and failure detector (Phi accrual) let nodes discover cluster state and detect failed nodes quickly.
- **Replication:** each partition is stored on RF nodes (e.g., RF=3) determined by the replication strategy (SimpleStrategy, NetworkTopologyStrategy across racks and datacenters). If one replica is down,

others still serve data.

- **Tunable consistency:** with RF=3, QUORUM needs 2 replicas, so one node may fail and the operation still succeeds; ONE succeeds even with two nodes down; LOCAL_QUORUM keeps operations inside a datacenter, surviving remote DC outages.
- **Hinted handoff:** when a replica is down, the coordinator stores hints and replays them when the node returns.
- **Read repair and anti-entropy repair (Merkle trees)** restore consistency among replicas.
- **Write path durability:** commit log + memtable + SSTables, so writes are never lost after a crash.

Trade-off: lower consistency levels give higher availability and lower latency but risk stale reads; higher levels (ALL) give stronger consistency but the operation fails if any replica is unavailable.

8. Explain the major query features of HBase and Cassandra. Compare HBase operations with Cassandra's CQL (Cassandra Query Language). Why is query design and data modeling particularly important in column-family databases?

HBase query features

- Basic operations: **Put, Get, Scan, Delete, Increment** via shell, Java API or REST/Thrift.
- Row-key lookups and range scans (STARTROW/STOPROW), filters (prefix, column, value filters), version and time-range retrieval.
- Coprocessors (server-side logic); SQL support through Apache Phoenix or Hive. No native joins or secondary indexes.

Cassandra query features

- **CQL** - SQL-like syntax: CREATE KEYSPACE/TABLE, INSERT, SELECT, UPDATE, DELETE, BATCH.
- Queries must specify the partition key; clustering columns give ordered range queries within a partition.
- Secondary indexes, materialized views, collections, user-defined types, lightweight transactions, TTL, counters. No joins, limited aggregates.

Task	HBase shell	Cassandra CQL
Create	create 'student','info'	CREATE TABLE student(id int PRIMARY KEY, name text, city text);
Insert	put 'student','S1','info:name','Anil'	INSERT INTO student(id,name) VALUES(1,'Anil');
Read one row	get 'student','S1'	SELECT * FROM student WHERE id=1;
Read many	scan 'student',{STARTROW=>'S1',LIMIT=>10}	SELECT * FROM student LIMIT 10; (or by partition)
Delete	delete 'student','S1','info:name'	DELETE name FROM student WHERE id=1;

Importance of query-first modeling: both systems lack joins and efficient ad-hoc filtering, and data is physically organised by key. A query that does not match the row key / partition key triggers full scans or is rejected. Therefore designers list the queries first, then create one denormalised table per query pattern, choosing keys that distribute data evenly and support required ranges. Wrong models cannot be fixed by indexes later and degrade performance and scalability.

9. Scaling is one of the major reasons for using NoSQL databases. Explain how HBase and Cassandra scale when data and user requests increase. Compare horizontal scaling, partitioning, replication, and load distribution in both systems.

Aspect	HBase	Cassandra
Horizontal scaling	Add RegionServers (and HDFS DataNodes); HMaster rebalances regions to them	Add nodes to the ring; data streams to new nodes automatically (bootstrap); no downtime

Aspect	HBase	Cassandra
Partitioning	Range partitioning by row key into Regions; auto-split when a region exceeds size limit (pre-splitting possible)	Hash partitioning of the partition key using consistent hashing (Murmur3) with virtual nodes
Replication	Done by HDFS (default 3 copies of blocks); one RegionServer serves a region at a time; optional cluster-to-cluster replication	Done by Cassandra itself; RF replicas per partition; NetworkTopologyStrategy for racks/DCs; all replicas serve requests
Load distribution	HMaster's balancer moves regions; hot row-key ranges need salting/pre-splitting	Hashing spreads data uniformly; coordinator routes requests with token-aware drivers
Bottlenecks	HMaster, ZooKeeper, HDFS NameNode (control plane)	Hot partitions, very wide partitions, compaction load

Summary: Both scale linearly by adding commodity machines. HBase uses range partitioning, which favours sequential scans but risks hotspots; Cassandra uses hash partitioning, which gives even distribution and simple elasticity but makes cross-partition range queries inefficient.

10. An organization wants to build a real-time event logging system that generates millions of events per minute. Explain why a column-oriented NoSQL database may be suitable for this application. Design a conceptual data model and discuss the required consistency, availability, and scaling strategy.

Why column-oriented NoSQL is suitable

- Log-structured storage (commit log/WAL + memtable + sequential flush) gives extremely high, append-style write throughput.
- Horizontal scaling absorbs millions of events per minute; sparse, evolving event attributes need no schema migration.
- Time-ordered data, built-in TTL and compression suit logs; efficient range queries by source and time.

Conceptual data model (Cassandra style)

```
CREATE TABLE events_by_source (
  source_id   text,
  day         date,          -- time bucket
  event_time  timeuuid,
  event_type  text,
  severity    text,
  payload     map<text,text>,
  PRIMARY KEY ((source_id, day), event_time)
) WITH CLUSTERING ORDER BY (event_time DESC)
  AND default_time_to_live = 2592000;   -- 30 days
```

Partition key = (source_id, day) keeps partitions bounded and evenly distributed; clustering key event_time stores events in time order. For HBase the equivalent row key is `salt#sourceId#reverseTimestamp` with families *meta* and *data*. Additional tables (e.g., events_by_type) can serve other queries.

Consistency, availability and scaling

- **Consistency:** eventual / low consistency is acceptable for logs (write with ONE or LOCAL_ONE; QUORUM for critical audit events). Events are immutable, so conflicts are rare.
- **Availability:** RF=3 across racks/DCs, masterless design, hinted handoff; the system keeps accepting writes during node failures.
- **Scaling:** add nodes as volume grows; use time-bucketed partitions, batching/async writes, token-aware drivers; TTL and TimeWindowCompactionStrategy keep storage under control; stream to Kafka/Spark for real-time analytics.

11. Content Management Systems (CMS) and Blogging Platforms often contain articles, authors, tags, comments, metadata, and user activity. Explain how a column-family database can be used to design such systems. Discuss the challenges

of handling varying and evolving data structures.

In a column-family store the design is driven by page/screen queries, with one denormalised table per access pattern.

Table (row key)	Column families / contents
articles (articleId)	content: title, body, status; meta: author_id, created, category, SEO fields; tags: tag_<name>=1; stats: views, likes (counters)
comments (articleId + commentTime)	One row (or clustering row) per comment: user, text, parent_id; fetching all comments of an article is a single range scan
articles_by_author (authorId + time)	Article ids/titles for the author page
articles_by_tag (tag + time)	Article ids for a tag page (tag-to-article inverted table)
users (userId)	profile, preferences, activity (recent reads, likes)

Benefits

- Articles of different types (blog, news, video) can have different attributes using dynamic qualifiers.
- Comments, tags and activity grow independently without table redesign; huge read/write scale for popular content; versioning keeps article revisions.

Challenges of varying and evolving structures

- No joins, so tag/author data are duplicated and must be kept in sync on update.
- Without schema enforcement, inconsistent qualifier names and data types can appear; applications need naming conventions and validation.
- Changing query patterns requires new tables and data migration/backfills.
- Old and new data formats coexist, so code must handle missing or legacy fields; wide rows (e.g., viral article comments) need bucketing.

12. Counters are frequently used in distributed applications for tracking page views, likes, downloads, API requests, and user activity. Explain how counters can be implemented in Cassandra. What problems can occur when counters are updated concurrently in a distributed environment?

Cassandra provides a special **counter** column type: a 64-bit value that can only be incremented or decremented, never directly set. Counter columns live in dedicated counter tables (all non-key columns must be counters).

```
CREATE TABLE page_stats (
  page_id text PRIMARY KEY,
  views   counter,
  likes   counter
);
UPDATE page_stats SET views = views + 1 WHERE page_id = 'p101';
SELECT views FROM page_stats WHERE page_id = 'p101';
```

Internally, each replica keeps its own shard of the counter and the total is the sum of shards. An update is a read-before-write on the leader replica which then replicates the new value. (In HBase the equivalent is the atomic `incr` / Increment operation on a cell.)

Problems with concurrent updates

- **Not idempotent:** if a write times out, the client cannot know whether it was applied; retrying may double-count, not retrying may lose the update, so counts are approximate.
- **Replica divergence and repair:** concurrent increments through different coordinators or during failures can leave replicas temporarily inconsistent.
- **Contention and hot counters:** a very popular counter becomes a hotspot because every increment goes through the same partition and requires a read-before-write (extra latency).

- **Limitations:** counters cannot be reset or set to a value, cannot have TTL, and cannot be mixed with regular columns; deletion and re-adding is unreliable.
- **Mitigations:** shard counters (e.g., page_id#bucket) and sum on read, use QUORUM consistency, batch increments in the application, or use an external stream processor when exact counts are required.

13. Explain the concept of expiring data (TTL - Time To Live) in Cassandra/HBase. Consider a system that stores temporary user sessions and automatically removes them after 24 hours. Explain how TTL can be useful and discuss its impact on storage management and application design.

TTL specifies the lifetime of data in seconds. After it passes, the data is treated as deleted and is physically removed later by compaction. In Cassandra, TTL can be set per INSERT/UPDATE (per cell) or as a table default (`default_time_to_live`). In HBase, TTL is configured per column family (`TTL => 86400`) and individual cells can carry a TTL attribute; expired cells are dropped during compaction.

```
-- Cassandra: session expires after 24 hours
INSERT INTO sessions (session_id, user_id, data)
VALUES ('abc123', 42, '...') USING TTL 86400;

# HBase
create 'sessions', {NAME => 'd', TTL => 86400}
```

Usefulness

- Automatic cleanup with no delete jobs; ideal for sessions, OTPs, caches, logs and temporary tokens.
- Keeps table size bounded, reducing storage cost and keeping reads fast.

Impact on storage management and design

- Space is not freed instantly: expired data becomes a tombstone (Cassandra) and is purged only on compaction after `gc_grace_seconds`; HBase drops it at compaction.
- Many tombstones can slow reads; use TimeWindowCompactionStrategy so whole SSTables expire together.
- Each update with a TTL resets the lifetime of that cell only; to extend a session the application must rewrite all columns with a new TTL.
- Applications must handle missing (expired) data gracefully and should not rely on exact-second expiry; clocks must be synchronised.

14. A company is developing a real-time analytics platform that must handle high write volume, large datasets, geographically distributed users, and occasional node failures. Compare HBase and Cassandra for this scenario and recommend one database. Justify your answer using architecture, consistency, availability, scalability, and query requirements.

Requirement	HBase	Cassandra
High write volume	Good (WAL + MemStore), but writes to a region go to one RegionServer	Excellent; any node accepts writes, writes spread evenly by hash
Large datasets	Excellent, stored in HDFS; strong for batch analytics with Hadoop/Spark	Excellent; petabyte scale on commodity nodes
Geo-distributed users	Weak: single-master style region ownership, cross-DC replication is asynchronous and complex	Native multi-DC replication, LOCAL_QUORUM for low latency in each region
Node failures	Region unavailable until reassigned; depends on ZooKeeper, HMaster, NameNode	No single point of failure; hinted handoff, replicas keep serving

Requirement	HBase	Cassandra
Consistency	Strong (CP)	Tunable (AP, can reach strong via QUORUM)
Query needs	Row-key scans, good Hadoop/Phoenix integration	Query-first CQL tables, Spark connector, time-series friendly

Recommendation: Apache Cassandra. Its masterless, multi-datacenter architecture handles high write volume and geographically distributed users with low latency; replication and tunable consistency keep the platform available through node failures; and scaling is linear by adding nodes. Analytics queries are supported by designing query-specific tables and using Spark for heavy aggregations. HBase would be chosen only if strict consistency and tight integration with an existing Hadoop ecosystem were the dominant requirements.

15. "Column-oriented NoSQL databases provide scalability and flexibility, but they require careful data modeling." Critically analyze this statement using HBase and Cassandra as examples. Discuss how poor choices of partition keys, row keys, column families, replication, and query patterns can negatively affect system performance.

The statement is accurate. The same design that gives these systems scalability (data distributed by key, no joins, no global optimizer) means the data model *is* the performance plan. Flexibility (dynamic columns, no rigid schema) removes database-enforced discipline, so mistakes in modeling go undetected until the system is under load, and fixing them often requires migrating data. Unlike an RDBMS, there is no query planner to compensate for a poor design.

Poor choice	Effect on HBase	Effect on Cassandra
Bad row/partition key	Monotonic keys (timestamp, sequential id) send all writes to one region/RegionServer = hotspot; very long keys waste memory	Low-cardinality key (e.g., country) or unbounded partition (all events of a sensor) creates hot, huge partitions, slow reads, compaction pressure
Too many column families	Each has its own MemStore and HFiles, causing excess flushes/compactions and memory pressure (recommended 2-3)	N/A (columns are in one table); but too many tables increase memory overhead
Wrong replication	HDFS replication of 1 risks data loss; excessive replication wastes storage and I/O	RF too low reduces availability; RF too high raises write cost; wrong strategy (SimpleStrategy across DCs) breaks locality and failure tolerance
Query patterns not matched to keys	Scans without a row-key prefix become full-table scans; filters are applied after reading data	Queries without the partition key need ALLOW FILTERING or secondary indexes, scanning many nodes; multi-partition queries have high latency
Consistency/other	Large cells and many versions bloat HFiles	Tombstones from deletes/TTL, wide rows, and unbatched or logged multi-partition batches degrade performance

Conclusion

Good modeling means: define the queries first, choose high-cardinality and well-distributed keys (salting or time-bucketing where needed), keep partitions and rows bounded, limit column families, denormalise per query, set replication and consistency to match availability needs, and test with realistic workloads. Scalability and flexibility are therefore only realised when the model is designed carefully.